

fdb-BM-test

The fdb-BM-test is used to simulate the quality and quantity of filesystem I/O operations in ECMWF's time-critical workflows.

The test writes synthetic forecast output fields from a number of concurrent writer processes executing on compute nodes.

At the same time, the test also simulates, concurrent to the aforementioned ongoing writing, the consumption of such recently written individual forecast fields by a number of reader processes fetching these fields for "product generation", "pgen" post-processing tasks; the pgen reader processes execute on different compute nodes than the writer processes, with the latter being part of each ensemble forecast member. For this, fdb-BM-test uses the same write and read methods from the fdb library, the ECMWF fields database <https://github.com/ecmwf/fdb>, as the production set-up.

Installation

```
Clone the code form the GitHub repository and then checkout fdb-BM-test_v.1.0 ""bash git clone
https://github.com/ecmwf/fdb-hammer-parallel.git cd fdb-hammer-parallel git checkout fdb-BM-test_v.1.0
""
```

Setup some build and install directories

```
# where the benchmark source will be cloned and the binaries installed
export install_dir=/hpc/file/system/install_dir

# fast file system where the benchmark will be built before installing
build_root=/tmp/fdb-hammer-parallel_${USER}

# file system where FDB will store the data
fdb_root=/hpc/file/system/fdb_root

# directory (shared or not) in the compute nodes where the benchmark and
# artifacts will be installed
artifact_dir=${install_dir}/fdb-hammer-parallel/artifacts

# create directories

mkdir -p $artifact_dir
mkdir -p $fdb_root
mkdir -p $TMPDIR
```

Obtain the fdb source, build and install the binaries

```
./setup.sh \
  --root $build_root \
```

```
--backend lustre \
--fdb-root $fdb_root
```

Runnnng fdb-BM-test

TODO add details of how the code works, including sending date to WRITERS and READERS

Setting up the WRITERS and READERS

The fdb-BM-test has been tested using the **nodeset** linux utility and the instructions below assume its availability. If **nodeset** isn't available or you have a different preferred utility, please consult its' instructions for functionality.

```
# Get the complete list of all nodes
ALL=$JOB_NODELIST # Adjust for the scheduler used

# Slice a single node off for the pre-liminary installation run
ONE=$(nodeset --slice 1 -f $ALL)
```

The test is best run when the **WRITERS** are evenly distributed across the available nodes, as this replicates the way an operational ensemble is run. This is not a requirement for testing but could be for acceptance testing.

Below are three different ways to split the available nodes across the **WRITERS** and **READERS**.

Splitting across separate groups of coupled nodes

If you have a situation where the nodes on the cluster are more connected for certain groups (e.g. nodes all on a single switch or within a leaf of a dragongly topology), then the techniques below can be used to split the nodes in the **WRITERS** and **READERS**.

Here it is assumed that there are 5 groups of nodes and that there are 150 writer nodes you want to split so that there the same number of **WRITERS** per group:

```
NUMBER_OF_GROUPS=5
NUMBER_OF_WRITER_NODES=150 # Must be a multiple of the number of groups

# Calculate the number of writers per group
WRITERS_PER_GROUP=$(( $NUMBER_OF_WRITER_NODES / $NUMBER_OF_GROUPS ))

# Expand ALL to full list of node names
ALL_NODES=$(nodeset -e "$ALL")

# Convert to array
NODE_ARR=$(echo $ALL_NODES | sed 's:,: :g')

NODE_ARRAY=( $NODE_ARR)
```

```
# Initialize groups
GROUP_A=()
GROUP_B=()

# Create associative array to track counts per prefix
declare -A PREFIX_COUNTS

# Loop through all nodes
for NODE in "${NODE_ARRAY[@]}; do
    PREFIX=${NODE:0:3} # Adjust if needed for your naming scheme
    COUNT=${PREFIX_COUNTS[$PREFIX]:-0}
    if [[ $COUNT -lt $WRITERS_PER_GROUP ]]; then
        GROUP_A+=("$NODE")
    else
        GROUP_B+=("$NODE")
    fi
    PREFIX_COUNTS[$PREFIX]=$((COUNT + 1))
done

# Convert to nodeset format
WRITERS=$(printf "%s\n" "${GROUP_A[@]}" | nodeset -f)
READERS=$(printf "%s\n" "${GROUP_B[@]}" | nodeset -f)
```

Splitting nodes evenly when writer nodes equals reader nodes

In the situation where the same number of writer nodes and reader nodes are used, and where these are situated is of no concern:

```
# Use nodeset split function
WRITERS=$(nodeset --split 2 -f $ALL | head -n 1)
READERS=$(nodeset --split 2 -f $ALL | tail -n 1)
```

Splitting nodes when uneven numbers of writer nodes and reader nodes

The below command picks the writer nodes randomly across all the available nodes and then splits the rest to reader nodes.

```
writernodes=$(( $writer_nodes_per_member * $num_members ))

WRITERS=$(nodeset -f --pick=$writernodes "$ALL")
READERS=$(nodeset -f "$ALL" -x "$WRITERS")

# If there are more available nodes for readers than are required then use
the pick command again to make READERS even
wanted_readers=$(( $reader_node_sets * $reader_nodes_per_step ))
READERS=$(nodeset -f --pick=$wanted_readers "$READERS")
```

Preliminary runs

Run a small single-process run to install binaries to compute nodes. It also acts as a sanity check to ensure the code is working.

```
./fdb-hammer.sh write \
  --nodelist $ONE --ppn 1 \
  --nmembers default --nsteps 10 --nlevels 1 --nparams 1 \
  --root $build_root --config $build_root/config.yaml.in \
  --artifact-dir $artifact_dir --artifact-dir-is-shared \
  --install --verbose

# Clean the FDB afterwards
rm -rf ${fdb_root?}/rd:xxxx:enfo:20230713:0000:g
```

It is recommended to do this before every main run to ensure the binaries are in the right place.

NOTE: if `--install` and `--artifact-dir-is-shared`, and `--nodelist` has more than one node, all nodes other than the first may fail due to libraries not being found

Main runs

```
# --- Setup the arguments to run - must be identical for write and read
hammer_args="--nodelist $WRITERS --ppn $writeppn \
  --nodelist-read $READERS --ppn-read $readppn --read-nodes-per-step
$rnpm \
  --nmembers $members \
  --root $build_root --config $build_root/config.yaml.in \
  --artifact-dir $artifact_dir"

first_step_complete=$(( $members * $memberdelay + $stepwindow + 20 ))
```

NOTE: The number of writer nodes per member is implied by the size of `$WRITERS` divided by `$members`

By default `memberdelay=2` and `stepwindow=10`

```
# --- run contending writers and readers

./fdb-hammer.sh write \
  $hammer_args \
  > write.out < /dev/null &

sleep $first_step_complete

./fdb-hammer.sh read \
  $hammer_args$ \
```

```
> read.out < /dev/null &

wait

# OPTIONAL - clean the fdb
rm -rf ${fdb_root?}/rd:xxxx:enfo:20230713:0000:g
```

Changing the FDB directory for a particular run

If there is a requirement to use multiple filesystems or pools for testing, the `setup.sh` script needs to be run each time to set the desired FDB location

```
new_fdb_root=/path/to/new/fdb_root
./setup.sh --root $build_root --backend lustre --fdb_root $new_fdb_root
```

Running Consistency Checks

TODO: Fix errors that prevent this from working at the moment

It is important that, not only, that the storage subsystem is able to manage the demands of the ECMWF operational workflow but also that it does so whilst ensuring the correctness of data.

Therefore, it is also required to do a separate run with consistency checks enabled that ensures the correctness of data is maintained. Enabling these checks affects performance and so no timing data is required for these runs.

```
# To enable consistency checks change the hammer_args to the following:
hammer_args="--nodelist $WRITERS --ppn $writeppn \
  --nodelist-read $READERS --ppn-read $readppn --read-nodes-per-step \
  $rnpn \
  --nmembers $members \
  --root $build_root --config $build_root/config.yaml.in \
  --artifact-dir $artifact_dir --no-itt --full-check"
```

NOTE: This disables the random ordering of a ITT benchmark run and makes the reading deterministic

Clean-up procedure

If the benchmark run hangs or terminates abruptly follow these steps to clean up. Uses the `clush` commands from `clustershell`

```
# list active writer/reader processes on the compute nodes
clush -w $ALL "ps -aux | grep '^$USER ' | wc -l"
# if any, kill as follows
#clush -w $ALL "ps -aux | grep 'fdb-hammer ' | awk '{print \$2}' | xargs -
```

```
I{} kill {}"
#clush -w $ALL "ps -aux | grep 'bash -s ' | awk '{print \$2}' | xargs -I{}
kill {}"
#clush -w $ALL "ps -aux | grep 'timeout 500' | awk '{print \$2}' | xargs -
I{} kill {}"

# list active orchestrating processes on the login/orchestrating node
ps -aux | grep fdb-hammer.sh
# if any, kill as follows
#ps -aux | grep fdb-hammer.sh | awk '{print \$2}' | xargs -I{} kill {}

# list leftover lock files in the compute nodes
clush -w $ALL 'ls /tmp/$USER/'
# if any, remove as follows
#clush -w $ALL 'rm -rf /tmp/$USER/fdb-hammer*'

# --- other cleanup tasks, usually required if needing to start from
scratch,
#     rerun with manipulated artifacts, or recompile the benchmark.
#     Sorted from less to more destructive.

# removes the FDB data directory
#rm -rf ${fdb_root?}

# removes the artifacts installed by setup.sh
#rm -rf ${SCRATCH?}/fdb-hammer-parallel

# removes the benchmark build
#rm -rf ${build_root?}

# removes the benchmark repository
#rm -rf ${SCRATCH?}/git/fdb-hammer-parallel
```